Copyright All rights reserved

08. 배열과 문자열
(Array – String)

Heesung Oh
https://github.com/3dapi

Heesung Oh
Copyright All rights reserved

- 배열(Array)
  - 같은 타입 데이터를 연속된 메모리 공간에 저장하는 구조
  - 인덱스로 직접 접근(random access)
    접근 속도 일정 → O(1). 가장 빠른 접근 자료 구조
  - sizeof → 배열 전체 크기 확인

- 특징
  - 크기: 컴파일 시점에 결정 → 실행 중 변경 불가
  - 지역 배열 → 스택 크기 제한
  - 전역/static → 정적 영역

Heesung Oh
Copyright All rights reserved

- 동적 메모리와 비교

구분	정적 배열	동적 메모리
크기 결정	컴파일 시 결정	실행 중 결정
메모리 위치	스택 / 정적 영역	힙
메모리 구조	연속된 메모리	연속된 메모리 블록
메모리 관리	없음	직접 관리(할당/해제 필수)
단편화	없음	발생 가능
크기	고정	변경 가능

Heesung Oh
Copyright All rights reserved

- **원소 주소 계산**
  - 원소 주소 = 배열_이름 + 인덱스
  - 바이트 관점: 주소 = 시작주소 + (인덱스 * sizeof(자료형))
    포인터 연산에서는 자료형 크기가 자동으로 반영

```
arr = 1000번지 (int = 4바이트)

      | arr[0] | arr[1] | arr[2] | arr[3] |

주소:   1000     1004     1008     1012

계산 방식:
arr[index] = base + (index * sizeof(int))
```

8.1 1차원 배열

Heesung Oh
Copyright All rights reserved

- 1차원 배열
  - 배열의 가장 기본적인 형태
  - 동일 자료형 데이터. 연속된 메모리 공간 저장 구조
  - 하나의 이름으로 다수 값 관리. 반복문과 결합 시 활용성 증가

- 1차원 배열 선언

```
자료형 배열_이름[배열요소_개수_상수];
```

  - 자료형: 저장 데이터 타입
  - 배열_이름: 식별자, 대부분 배열 시작 주소 ↔ `&arrayVal[0];`
  - size : 배열 원소(element) 개수, 컴파일 시점에 결정되는 상수 표현식

```
배열_이름 ↔ &arrayVal[0];
```

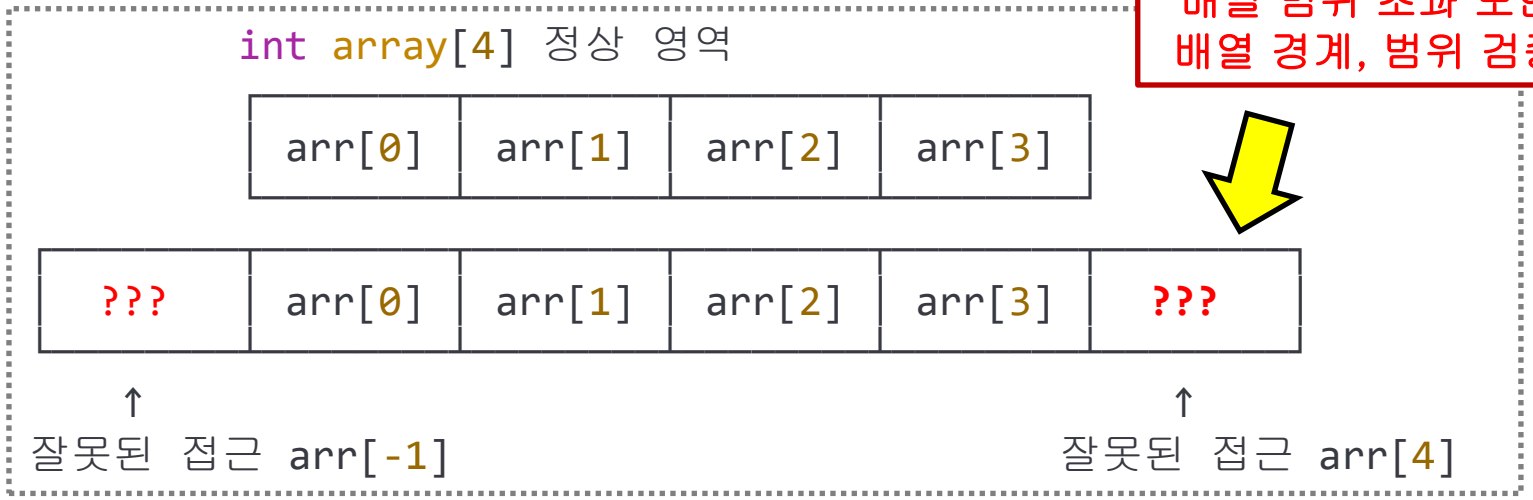
Heesung Oh
Copyright All rights reserved

- **배열 접근**
  - 원소 접근 유효 인덱스: [0, 크기 -1]

```
배열과_동등한_자료형 value = 배열_이름[인덱스];
배열_이름[인덱스] = 값;
```

- **범위를 벗어난 접근(Undefined Behavior)**

버퍼 오버플로
배열 범위 초과 보안 취약
배열 경계, 범위 검증 필수



8.1.2 1차원 배열 선언과 동시에 초기화

Heesung Oh
Copyright All rights reserved

- **배열 선언 + 초기화 동시 가능**

```
int arr[32] = {10, 20, 30, 40}; // 32개 원소 중 4개만 초기화
int arr[] = {10, 20, 30, 40, 50}; // 원소 개수: 5
```

- **지정한 값으로 원소 초기화. 나머지는 0 으로 자동 초기화**

arr[0] = 10, arr[1] = 20, arr[2] = 30, arr[3] = 40,
arr[4] ~ arr[31] = 0

- **배열 초기화 →모든 요소 0 초기화**

```
int arr[32] = {0};
```

- **배열 크기 없이 배열 선언 초기화**
  - **원소 개수: 초기화 하고 있는 원소 개수**

8.1.3 1차원 배열의 크기와 원소의 개수

Heesung Oh
Copyright All rights reserved

```
배열 원소 크기 = sizeof(배열_이름[0])
배열 전체 크기 = sizeof(배열_이름)
배열 요소 개수 = 배열 전체 크기 / 배열 원소 크기
```

```
int arr[128];
sizeof(arr);          // 512 (int가 4바이트 가정)
sizeof(arr[0]);       // 4바이트
int arr_number = sizeof(arr) / sizeof(arr[0]); // 128
```

8.1.4 1차원 배열 접근 예제

Heesung Oh
Copyright All rights reserved

```c
#include <stdio.h>

int main(void)
{
    int arr[100] = {0};   // 배열 원소를 0으로 초기화
    int arr_number = sizeof(arr) / sizeof(arr[0]);

    // 배열에 1부터 100까지 값 저장
    for(int i = 0; i < arr_number; ++i)
    {
        arr[i] = i + 1;
    }

    // 합계 계산
    int sum = 0;
    for(int i = 0; i < arr_number; ++i)
    {
        sum += arr[i];
    }

    // 평균 계산    및 출력
    double average = (double)sum / arr_number;
    printf("평균: %.4f\n", average);

    return 0;
}
```

8.1.5 1차원 배열의 최대값 / 최소값 찾기

Heesung Oh
Copyright All rights reserved

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    srand((unsigned int)time(NULL));

    // 배열 선언 및 초기화.
    int arr[128] = {0};
    int arr_count = sizeof(arr) / sizeof(arr[0]);

    // 난수로 배열 채우기 [0,999]
    for (int i = 0; i < arr_count; ++i)
    {
        arr[i] = rand() % 1000;
    }
```

```c
    // ...
    int maxVal = arr[0];    // 최대값 저장 변수 초기화
    int minVal = arr[0];    // 최소값 저장 변수 초기화
    for (int i = 1; i < arr_count; ++i)
    {
        // 배열 원소 값이 최대값 보다 크면 저장
        if (arr[i] > maxVal)
            maxVal = arr[i];

        // 배열 원소 값이 최소값 보다 작으면 저장
        if (arr[i] < minVal)
            minVal = arr[i];
    }

    printf("최대값: %d\n", maxVal);
    printf("최소값: %d\n", minVal);

    return 0;
```

Heesung Oh
Copyright All rights reserved

- 양수, 음수, 짝수/홀수 카운트
  - 난수 [-5000, 5000]

```c
srand((unsigned int)time(NULL));

// 배열 선언 및 초기화.
int arr[128] = {0};
int arr_count = sizeof(arr)/sizeof(arr[0]);

// 난수로 배열 채우기 [-5000, 5000]
for (int i = 0; i < arr_count; ++i)
{
  arr[i] = -5000 + (rand() % (10000 + 1));
}
```

```c
int positive_count = 0;     // 양수 개수
int negative_count = 0;     // 음수 개수
int even_count = 0;         // 짝수 개수
int odd_count = 0;          // 홀수 개수

for (int i = 0; i < arr_count; ++i)
{
    if (arr[i] > 0)         // 0과 비교 양수
        ++positive_count;
    else if (arr[i] < 0)    // 0과 비교 음수
        ++negative_count;

    if (arr[i] % 2 == 0)    // 나머지 연산자로 짝수 확인
        ++even_count;
    else
        ++odd_count;
}

printf("양수 개수: %d\n", positive_count);
printf("음수 개수: %d\n", negative_count);
printf("짝수 개수: %d\n", even_count);
printf("홀수 개수: %d\n", odd_count);
```

8.1.7 1차원 배열 역순 출력(Reverse)

Heesung Oh
Copyright All rights reserved

- 배열 역순
  - 시작: (N – 1), 종료: 0, 방향: 감소(--)

```
int arr[32] = {0};
int arr_count = sizeof(arr) / sizeof(arr[0]);

// 배열에 1부터 32까지 값 저장
for (int i = 0; i < arr_count; ++i)
{
    arr[i] = i + 1;
}

// 배열을 역순으로 출력
for (int i = arr_count - 1; i >= 0; --i)
{
    printf("%d ", arr[i]);
}
```

8.1.8 1차원 배열 복사(Copy)

Heesung Oh
Copyright All rights reserved

- **배열 복사**
  - 배열 이름 = 배열 이름; → 복사 불가 (배열 이름 → 주소 개념)
    - 배열 전체 복사 문법 없음
    - 요소 단위 복사 필요 → 반복 구조 사용
  - 순방향 / 역순 복사 가능:
    - 데이터 덮어쓰기 방지 → 역순 복사 사용
  - 결과: 값 동일 / 메모리 독립

```
// 올바른 복사 방법: 원소를 하나씩 복사
for (int i = 0; i < arr_count; ++i)
{
    dst[i] = src[i];
}

// 뒤에서부터 복사. while(--i) 은 표현이 간결
int index = arr_count;
while (--index >= 0)
{
    dst[index] = src[index];
}
```

8.1.9 1차원 배열 원소의 값 교환(Swap)

Heesung Oh
Copyright All rights reserved

- 배열 원소 교환
  - 두 값 직접 교환 불가 → 임시 변수 필요
    - 값 이동 과정: A → temp, B → A, temp → B
    - 특정 인덱스 두 위치 선택 → 값 교환
  - 정렬 알고리즘 기본 동작

```
// 값 교환(Swap)
int temp = arr[a];
arr[a] = arr[b];
arr[b] = temp;
```

8.1.10 1차원 배열 정렬(Bubble Sort)

Heesung Oh
Copyright All rights reserved

- Bubble Sort
  - 인접한 두 원소 비교 → 순서 어긋남 → 자리 교환
  - 반복 수행 → 큰 값 오른쪽 이동: 거품처럼 위로 이동 → Bubble Sort
  - 단순 구조
    - 성능 낮음 → 교육용
    - 기본 개념 정렬된 자료 O(n). 정렬되지 않은 자료 O(n^2)
- 동작 흐름
  - 한 번 반복 → 가장 큰 값 맨 뒤 이동
  - 반복 횟수 증가 → 정렬 범위 감소
  - 전체 반복 → 오름차순 정렬 완료

```
// 버블 정렬
for (int pass = 0; pass < arr_count - 1; ++pass)
    for (int i = 0; i < arr_count - 1 - pass; ++i)
        if (arr[i] > arr[i + 1])
        {
            int temp = arr[i];
            arr[i] = arr[i + 1];
            arr[i + 1] = temp;
        }
```

8.1.10 1차원 배열 정렬(Bubble Sort)

Heesung Oh
Copyright All rights reserved

- Bubble Sort
  - 인접한 두 원소 비교 → 순서 어긋남 → 자리 교환
  - 반복 수행 → 큰 값 오른쪽 이동: 거품처럼 위로 이동 → Bubble Sort
  - 단순 구조
    - 성능 낮음 → 교육용
    - 기본 개념 정렬된 자료 O(n). 정렬되지 않은 자료 O(n^2)
- 동작 흐름
  - 한 번 반복 → 가장 큰 값 맨 뒤 이동
  - 반복 횟수 증가 → 정렬 범위 감소
  - 전체 반복 → 오름차순 정렬 완료

```
// 버블 정렬
for (int pass = 0; pass < arr_count - 1; ++pass)
    for (int i = 0; i < arr_count - 1 - pass; ++i)
        if (arr[i] > arr[i + 1])
        {
            int temp = arr[i];
            arr[i] = arr[i + 1];
            arr[i + 1] = temp;
        }
```

8.1.10 1차원 배열 접근 – 버블 정렬

Heesung Oh
Copyright All rights reserved

단계 swap: 노랑, 이번 단계 확정: 마젠타, 정렬 완료: 하늘색

0	9	4	7	1	5	10	2	7	6	4
1	4	7	1	5	9	2	7	6	4	10
2	4	1	5	7	2	7	6	4	9	10
3	1	4	5	2	7	6	4	7	9	10
4	1	4	2	5	6	4	7	7	9	10
5	1	2	4	5	4	6	7	7	9	10
6	1	2	4	4	5	6	7	7	9	10
7	1	2	4	4	5	6	7	7	9	10

8.1.11 1차원 배열 접근 – 삽입 정렬

Heesung Oh
Copyright All rights reserved

- Insertion Sort
  - 왼쪽 구간 → 항상 정렬 상태 유지
  - index 1부터 시작, 현재 값(key)을 왼쪽과 비교
  - key보다 큰 값 → 오른쪽으로 이동
  - 반복 후 빈 위치에 key 삽입

- 성능
  - 이미 정렬된 경우 → O(n)
  - 일반 경우 → $O(n^2)$

```
for(int i = 1; i < arr_count; ++i)
{
  int key = arr[i];     // 새로 삽입할 값
  int j = i - 1;

  // key보다 큰 값들을 한 칸씩 오른쪽으로 이동
  for( ;0 <= j && key < arr[j]; --j)
  {
      arr[j + 1] = arr[j];
  }

  // 비어 있는 위치에 key 삽입
  arr[j + 1] = key;
}
```

8.1.11 1차원 배열 접근 – 삽입 정렬

Heesung Oh
Copyright All rights reserved

단계 key: 붉은색, 이번 단계 확정: 마젠타, 정렬 완료: 하늘색

단계											
0	9		4	7	1	5	10	2	7	6	4
1	4	9		7	1	5	10	2	7	6	4
2	4	7	9		1	5	10	2	7	6	4
3	1	4	7	9		5	10	2	7	6	4
4	1	4	5	7	9		10	2	7	6	4
5	1	4	5	7	9	10		2	7	6	4
6	1	2	4	5	7	9	10		7	6	4
7	1	2	4	5	7	7	9	10		6	4
8	1	2	4	5	6	7	7	9	10		4
9	1	2	4	4	5	6	7	7	9	10	

Heesung Oh
Copyright All rights reserved

- **다차원 배열**
  - 2차원 이상 배열. 가장 많이 사용 → 2차원
  - n차원 → 인덱스 계산 통해 1차원으로 선형화 가능

- **선언 형식**

```
자료형 배열_이름[1차원_개수][2차원_개수]...[n차원_개수];
```

- **원소 접근**
  - 각 차원 인덱스 순서대로 접근

```
원소 주소 = (배열_이름
        + (...(((1차원_인덱스 × 2차원_개수 + 2차원_인덱스)
                              × 3차원_개수 + 3차원_인덱스)
                              × 4차원_개수 + 4차원_인덱스) ... )
                              × n차원_개수 + n차원_인덱스) * 자료형 크기;
```

8.2 다차원 배열

Heesung Oh
Copyright All rights reserved

- 메모리 구조
  - 물리 메모리 → 1차원 연속 공간
  - C 배열 → row-major order
  - 마지막 차원 인덱스가 가장 빠르게 증가

- 주소 계산
  - 배열 이름 → 0번째 원소 주소로 변환 (array-to-pointer decay)
  - 다차원 인덱스 → 1차원 인덱스로 환산
  - 선형 인덱스 = ((((i1 * D2 + i2) * D3 + i3) ...) * Dn + in)

- 1차원 변환
  - 동일한 인덱스 계산 필요

```
자료형 배열_이름[1차원_개수 * 2차원_개수 * ... * n차원_개수];
```

8.2.1 2차원 배열

Heesung Oh
Copyright All rights reserved

- 2차원 배열
  - 가장 많이 사용되는 형태
  - 1차원 메모리에 펼쳐 저장

- 선언 + 구조

```
자료형 배열_이름[행의_개수][열의_개수];
```

```
자료형 배열_이름[][열의_개수] =
                {
                    {각 행의 초기값},
                    {각 행의 초기값}, ...
                };
```

8.2.1 2차원 배열

Heesung Oh
Copyright All rights reserved

- **메모리 배치 (row-major)**

```
int arr[3][4]
```

```
[ arr[0][0], arr[0][1], arr[0][2], arr[0][3],
  arr[1][0], arr[1][1], arr[1][2], arr[1][3],
  arr[2][0], arr[2][1], arr[2][2], arr[2][3] ]
```

- **논리 구조 (2D)와 실제 메모리 (1D): 행 기준 연속 저장**

<논리 구조>

[0][0]	[0][1]	[0][2]	[0][3]
[1][0]	[1][1]	[1][2]	[1][3]
[2][0]	[2][1]	[2][2]	[2][3]

<실제 메모리>

00	01	02	03	10	11	12	13	...	22	23

8.2.1 2차원 배열

Heesung Oh
Copyright All rights reserved

- 행/열 개수 계산

```
int row_count = sizeof(arr    ) / sizeof(arr[0]);     // 행 개수
int col_count = sizeof(arr[0]) / sizeof(arr[0][0]); // 열 개수
```

- 전체 0 초기화 (zero initialization)

```
int arr2d[3][4] = {0};
```

- 1차원 나열 초기화 : 부족한 값 → 0 (zero initialization)

```
int arr2d[3][4] = {1, 2, 3, 4, 5, 6};
```

```
{1, 2, 3, 4}
{5, 6, 0, 0}
{0, 0, 0, 0}
```

- 행(row) 단위 초기화: 부족한 값 → 0 (zero initialization)

```
int arr2d[3][4] =
 {
   {1, 2, 3, 4},
   {7},
   {5, 6},
};
```

```
{1, 2, 3, 4}
{7, 0, 0, 0}
{5, 6, 0, 0}
```

8.2.1 2차원 배열 출력

Heesung Oh
Copyright All rights reserved

- i, j 사용

```
for(int i = 0; i < row_count; ++i)
{
    for(int j = 0; j < col_count; ++j)
    {
        printf("%3d ", arr[i][j]);
```

- x, y 사용
  - arr[y] [x] → y: 행, x: 열
  - 메모리 → 1차원 (row-major): ex) image
  - 좌표 기반 접근 → 직관적

```
for(int y = 0; y < row_count; ++y)
{
    for(int x = 0; x < col_count; ++x)
    {
        printf("%3d ", arr[y][x]);
```

Heesung Oh
Copyright All rights reserved

- 최대값 / 최소값 / 양수 / 음수 / 짝수 / 홀수

```
if (value > maxVal)
    maxVal = value;          // 최대값

if (value < minVal)
    minVal = value;          // 최소값

// 양수/음수
if (value > 0)
    ++positive;
else if (value < 0)
    ++negative;

// 짝수/홀수
(value % 2 == 0)? ++even: ++odd;
```

8.2.3 2차원 배열 ↔ 1차원 배열 변환

Heesung Oh
Copyright All rights reserved

- 변환
  - 2차원 배열 ↔ 1차원 배열
  - 2차원 배열 → 1차원 연속 메모리 (row-major)
  - 행 단위 저장 → 기준: 열 개수
    - index = i * col + j

- 2D → 1D
  - index1d = (행 인덱스 × 열 개수) + 열 인덱스
  - arr1d[i * col + j]

- 1D → 2D
  - 행 인덱스 = index / 열 개수
  - 열 인덱스 = index % 열 개수

8.2.4 2차원 배열 행과 열 전치

Heesung Oh
Copyright All rights reserved

- **2차원 행렬 전치 (Transpose)**

```
M[i][j] → M[j][i]
```

```
// 전치 (Transpose)
for(int r = 0; r < row_count; ++r)
{
  for(int c = 0; c < col_count; ++c)
  {
    result[c][r] = arr2d[r][c];
  }
}
```

8.2.5 3차원 배열

Heesung Oh
Copyright All rights reserved

- 면(layer, page) - 행(row) - 열(column) 구조

```
자료형 배열_이름[면_개수][행_개수][열_개수];
```

- 1차원 연속 메모리(row-major order)

```
int arr[2][3][4] = {0};
```

```
arr[0][0][0] arr[0][0][1] arr[0][0][2] arr[0][0][3]
arr[0][1][0] arr[0][1][1] arr[0][1][2] arr[0][1][3]
arr[0][2][0] arr[0][2][1] arr[0][2][2] arr[0][2][3]
arr[1][0][0] arr[1][0][1] arr[1][0][2] arr[1][0][3]
arr[1][1][0] arr[1][1][1] arr[1][1][2] arr[1][1][3]
arr[1][2][0] arr[1][2][1] arr[1][2][2] arr[1][2][3]
```

8.2.5 3차원 배열

Heesung Oh
Copyright All rights reserved

- 3D → 1D

```
인덱스 = ((면_인덱스 × 행 개수) + 행_인덱스)
          × 열_개수 + 열_인덱스;
```

- ex)

```
for(int l = 0; l < layer_count; ++l)
{
  for(int r = 0; r < row_count; ++r)
  {
    for(int c = 0; c < col_count; ++c)
    {
      arr[l][r][c]...
```

8.3 문자열 리터럴과 char 배열

Heesung Oh
Copyright All rights reserved

- 문자열 리터럴(String)
  - 문자열 = char 배열
  - 문자열 끝 → 널 문자 '\0' ← 값 0

```
'H', 'e', 'l', 'l', 'o', '\0'
```

```
"Hello\0"
```

⇨

```
'H', 'e', 'l', 'l', 'o', '\0', '\0'
```

- Not null-terminated string
  - 배열 끝에 널문자 없는 char 배열
  - 문자열 아님

```
char charArray[] = { 'H', 'e', 'l', 'l', 'o' };
```

⇨

```
{'H', 'e', 'l', 'l', 'o', '\0' };
```

```
{'H', 'e', 'l', 'l', 'o', 0 };
```

Heesung Oh
Copyright All rights reserved

- sizeof (문자열)
  - 문자 수 + 1 (널문자)

```
// 문자열 리터럴 → 자동으로 '\0' 포함
// 내부: 'H','e','l','l','o','\0' → 총 6바이트
char myStr1[] = "Hello";

// 문자열 리터럴 + 명시적 '\0' 추가
// 내부: 'H','e','l','l','o','\0','\0' → 총 7바이트
// 문자열 리터럴 자체에 이미 '\0'이 있기 때문에 하나 더 추가됨
char myStr2[] = "Hello\0";

// 포인터 → 문자열 리터럴을 가리킴
// 포인터 변수 자체는 문자열 크기가 아니라 주소 크기를 가짐
const char* myStr3 = "Hello";

printf("array(\"Hello\")   : %zu\n", sizeof(myStr1)); // 6
printf("array(\"Hello\\0\") : %zu\n", sizeof(myStr2)); // 7
printf("pointer(\"Hello\") : %zu\n", sizeof(myStr3)); // 8 (64bit 기준)
printf("literal(\"Hello\") : %zu\n", sizeof("Hello")); // 6 ('\0' 포함)
```

8.3 문자열 리터럴과 char 배열

Heesung Oh
Copyright All rights reserved

- 문자열 특징과 장점
  - 배열 크기 자동 계산
  - 널 문자 '\0' 자동 추가
  - '\0' 기준으로 처리
  - 문자열 함수 사용 가능 (strcpy, strcmp, strlen 등)
  - 문자열 리터럴 → 읽기 전용
  - char 배열 → 수정 가능
  - 널 문자 없으면 문자열 아님 → char 배열

8.3.1 함수에서 배열 인수 전달

Heesung Oh
Copyright All rights reserved

- 배열 전달 시
  - 포인터로 변환 (Array-to-pointer conversion, array-to-pointer decay)
    - 배열 전체 복사 ❌
    - 주소 전달 ⭕

    Ex) int arr[] → int*, char byteArray[] → char* byteArray

  - sizeof(arr) → 배열 크기 아닌 포인터 변수의 크기
  - 배열 크기 → 별도 전달 필요
  - 배열 매개변수([]) → 포인터(*)로 변환

```
void ModifyArray(int arr[], int size)
{
    for(int i = 0; i < size; ++i)
    {
        arr[i] = arr[i] * 2;
    }
}
```

8.3.1 함수에서 배열 인수 전달

Heesung Oh
Copyright All rights reserved

- 함수의 인수
  - 배열은 포인터로 변환되어 전달 (array-to-pointer conversion)
  - 전달 시 0-요소의 주소(&arr[0])로 변환
  - 포인터로 전달되므로 배열의 크기 정보는 전달 안됨

```
void Func(int arr[])
```

```
void Func(int arr[10000]) // 배열크기 정보는 전달 안됨
```

```
void Func(int* arr)
```

8.3.2 printf, scanf "%s" 이해

Heesung Oh
Copyright All rights reserved

- printf %s
  - 문자열 처리
  - 출력
    - 시작 주소부터 '\0' 까지 출력
  - 입력
    - 공백 전까지 읽음
    - 마지막에 '\0' 자동 추가

- scanf %s
  - 입력 → 버퍼 → 배열로 복사
  - 공백(스페이스, 탭, 엔터)에서 종료
  - 문자열 형태로 저장 ('\0' 포함)
  - 주의 사항
    - 배열 크기 초과 시 메모리 침범
    - 입력 길이 제한 필요

8.3.2 myScanf

Heesung Oh
Copyright All rights reserved

- 목적
  - 안전한 문자열 입력 (크기 제어, 버퍼 정리)
- 동작
  - getchar로 한 글자씩 입력, 줄바꿈까지 읽어 입력 버퍼 비움
  - 버퍼 크기 초과 방지, '\0' 직접 추가 → 문자열 종료 보장

```
int myScanf(char buf[], int bufsize)
{
    int read_count = 0; // 읽은 문자 개수.
    int index = 0;      // 배열 마지막에 널문자 추가하기 위해 밖으로 뺌.

    // 한 글자씩 읽어서 배열에 저장
    for(int ch = 0;(ch = getchar()) != '\n' && ch != EOF; ++index)
    {
        // overflow 방지. 마지막 한 칸은 널 문자용으로 남겨 둠.
        if(index < bufsize - 1) // 버퍼 크기 전까지만
        {
            buf[index] = (char)ch;
            ++read_count;
        }
    }
    buf[read_count] = '\0';     // 문자열 종료 보장.
    return read_count;          // 읽은 문자를 반환.
}
```

8.3.3 문자열 길이 - myStrLen

Heesung Oh
Copyright All rights reserved

- 문자열 끝 → '\0'
- 문자열 길이 → '\0' 이전까지

```
size_t myStrLen(char str[])
{
    if(NULL == str)
        return 0;   // 널 포인터면 길이는 0.

    size_t len = 0;
    // 널문자 전까지가 문자열의 길이.
    while(str[len] != '\0')
    {
        ++len;
    }
    return len;
}
```

8.3.4 문자열 복사하기 - myStrCpy

Heesung Oh
Copyright All rights reserved

- 문자열 끝 → '\0'
- 문자열 길이 → '\0' 이전까지

```
size_t myStrCpy(char dst[], size_t dest_size, const char src[])
{
    if(0 == dest_size)  // 크기 0이면 복사할 수 없으므로 0 반환.
        return 0;

    size_t n=0;
    --dest_size;        // 널문자 저장 공간을 위해 1 감소.
    for(;'\0' != src[n] && 0<dest_size; ++n, --dest_size)
    {
        dst[n] = src[n];
    }
    dst[n] = '\0';      // 널문자 저장.
    return n;
}
```

8.3.5 문자열 비교하기

Heesung Oh
Copyright All rights reserved

- 문자열 비교
  - 앞에서부터 문자 비교
  - 다른 순간 크기 결정
  - 끝까지 같으면 길이 비교
- 반환 규칙
  - s1 〉 s2 → +1
  - s1 == s2 → 0
  - s1 〈 s2 → -1

```
int myStrCmp(const char s1[], const char s2[])
{
    int i = 0;
    for(;s1[i] != '\0' && s2[i] != '\0'; ++i)
    {
        if (s1[i] != s2[i])
        {
            if(s1[i] > s2[i])
                return +1;
            else
                return -1;
        }
    }
    if(s1[i] == s2[i])
        return 0;
    else if(s1[i] > s2[i])
        return +1;
    return -1;
}
```

8.3.6 문자열 함수

Heesung Oh
Copyright All rights reserved

- 문자열 함수
  - 〈string.h〉 제공 함수 → strlen, strcpy, strcmp
- 문자열 구조
  - 문자 배열, 마지막 → '\0'
- 처리 방식
  - 앞에서부터 순차 접근
  - '\0' 까지 반복

6. 학습 정리

Heesung Oh
Copyright All rights reserved

- **배열**
  - 동일한 타입의 연속된 메모리 공간
  - 배열 이름 → 0 번째 원소의 주소, 배열 원소 → 배열이름[인덱스]
  - Index 기반 접근 → O(1) 시간
  - 크기: 컴파일 시점에서 결정, 범위 초과 → UB
  - 배열 → 포인터로 변환 (array-to-pointer decay)
  - 2차원, 3차원, n차원 → 1차원 메모리로 변환 가능
  - 저장 방식: row-major order

- **문자열**
  - 문자열 = '\0' 로 끝나는 char 배열
  - 리터럴 → 자동 '\0' 추가
  - sizeof → '\0' 포함
  - 문자열 처리 = 순차 탐색
  - '\0' 까지 반복
  - printf("%s") → '\0' 까지 출력
  - scanf("%s") → '\0' 자동 추가
  - 입력 시 배열 크기 초과 주의

실습 및 과제

Heesung Oh
Copyright All rights reserved

- 과제 1: 삽입 정렬
  - 정수 배열 100개 (1000~9999 난수)
  - 오름차순 / 내림차순 선택 ('a', 'd')
  - 하나의 함수에서 처리
  - 정렬 전 / 후 출력
  - 잘못된 입력 → 재입력

- 과제 2: 선택 정렬
  - 동일 조건
  - 최소/최대 선택 → 교환 방식
  - 오름차순 / 내림차순 처리

- 과제 3: 지형 출력
  - 32×32 배열 (0 / 1)
  - 이중 for문 사용
    - 1 → '#'
    - 0 → 공백

실습 및 과제

Heesung Oh
Copyright All rights reserved

- 과제 4: 아이콘
  - 16×16 → 1차원 배열
  - 출력 시 2차원 형태로 변환
  - index = row * 16 + col
  - 아이콘 3개 이상 출력

- 과제 5: 문자열 배열/숫자 배열 출력
  - 배열 4개
    - 이름: 문자열
    - 점수: 정수
    - HP: 실수
  - 길이: 5
  - while문 사용 출력